

Project 5

Lecturer: Shuai Wang

TA: Jiawei Liu

Requirements for submitting the assignment

- **Email address :** liujoy@seu.edu.cn
- **Format of Email subject and File name:**

StudentNumber-Name-Project-X

example : *71121000-张三-Project-5.zip*

- **Including :**

① *report_A5.doc / report_A5.docx / report_A5.pdf*

② *university_A5.mdb / university_A5.accdb*

Deadline: Beijing time, 12.18, 23:59:59

Project 5

I. Tutorial on Integrity Constraint

II. Assignment

Integrity Constraint

Other Integrity Constraints

- Domain integrity constraint
 - An attribute's value must be a value in the domain of this attribute. This is the most basic constraint. All popular RDBMS are able to check domain integrity constraint automatically.
- Entity integrity constraint
 - Every relation should have a primary key. The value of primary key of each tuple must be unique. Primary key cannot be *NULL*. This is so-called entity integrity constraint.

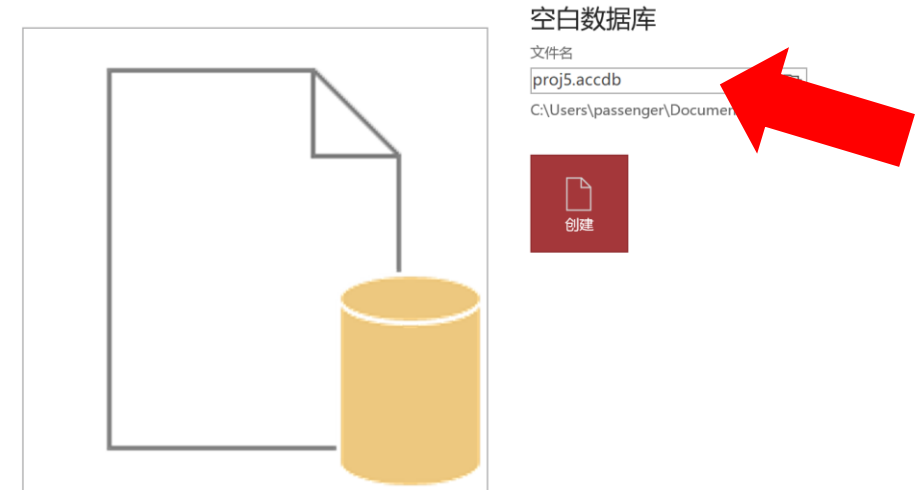
Foreign Keys, Referential Integrity

- *Foreign key* : Set of attributes in one relation that is used to 'refer' to a tuple in another relation. (Must correspond to primary key of the second relation.) Like a 'logical pointer'.
- E.g. *sid* is a foreign key referring to **Students**:
 - Enrolled(*sid*: string, *cid*: string, *grade*: string)
 - If all foreign key constraints are enforced, *referential integrity* is achieved, i.e., no dangling references.
 - Have you forgotten *soft link*?

Integrity Constraint

1. Preparation

1) Create Database



2) Create Table

•学生表 (students)

- 学号 (StudentID)
- 姓名 (Name)
- 年龄 (Age)
- 性别 (Gender)

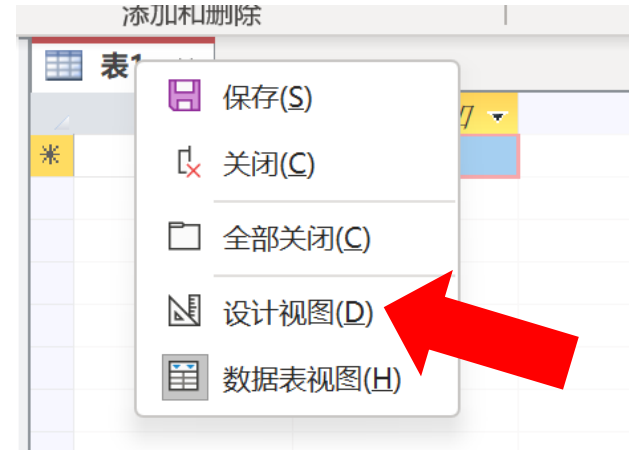
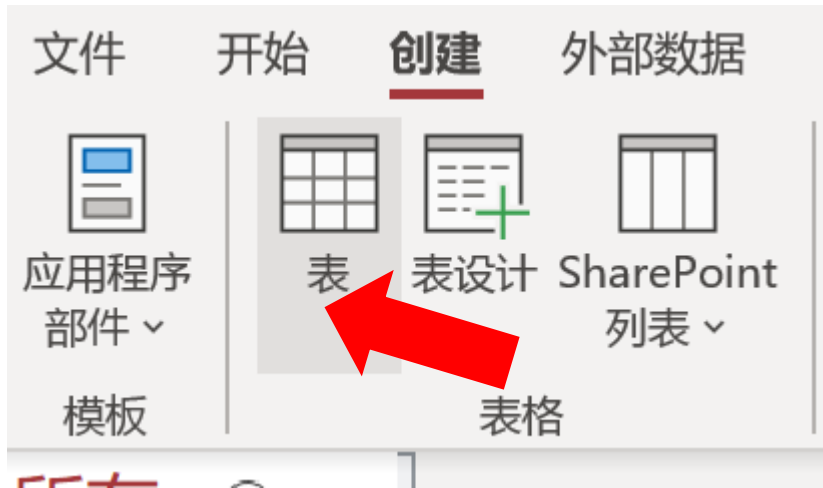
•课程表 (courses)

- 课程号 (CourseID)
- 课程名 (CourseName)
- 学分 (Credits)

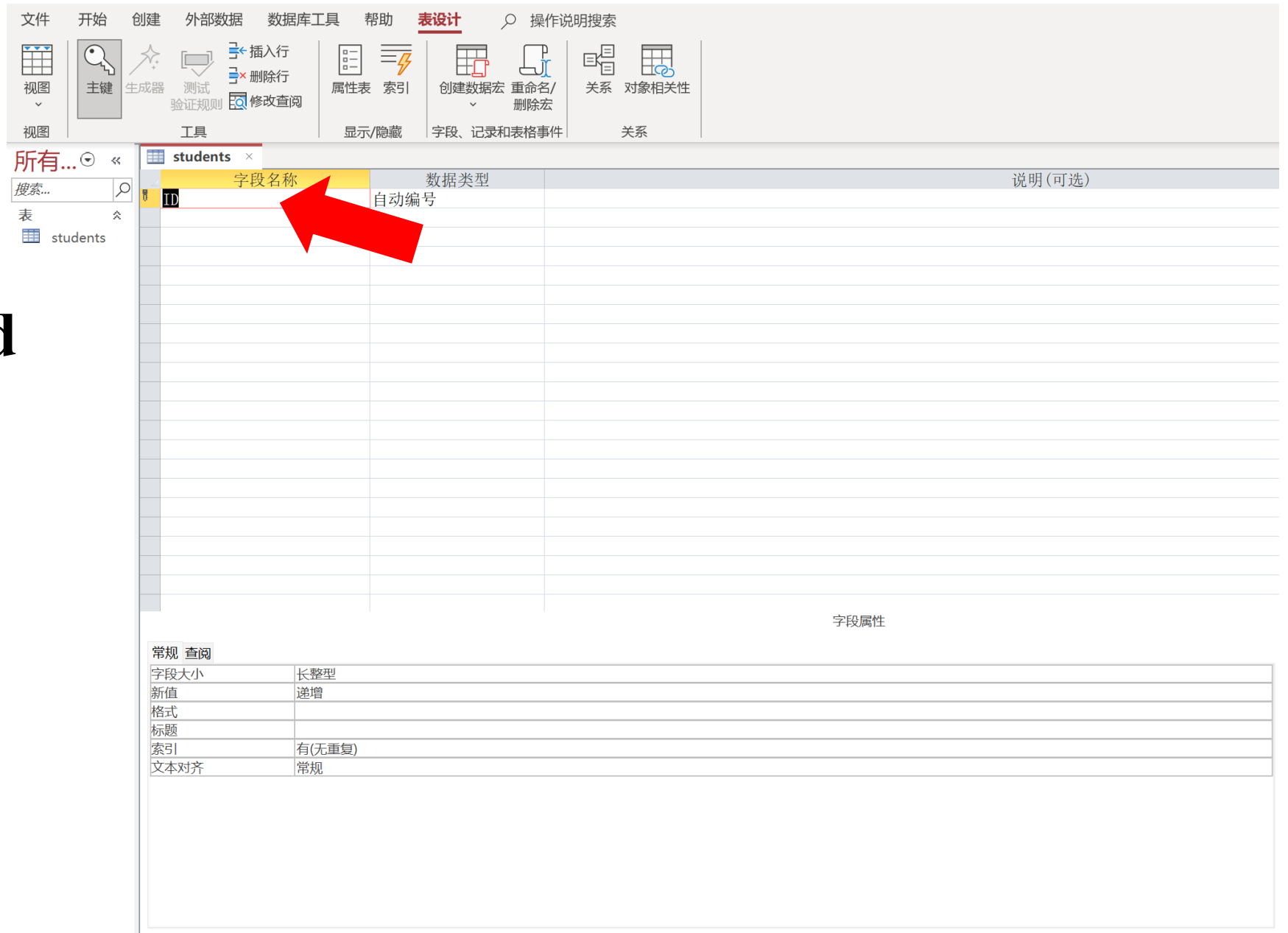
•选课表 (enrollments)

- 学号 (StudentID)
- 课程号 (CourseID)
- 成绩 (Grade)

Create “students” Table



Create “students” Table



3) Rename Field

•学生表 (students)

- 学号 (StudentID)
- 姓名 (Name)
- 年龄 (Age)
- 性别 (Gender)

Not “自动编号”

字段名称	数据类型
StudentID	短文本
Name	短文本
Age	数字
Gender	短文本

[Optional]: Set Primary Key

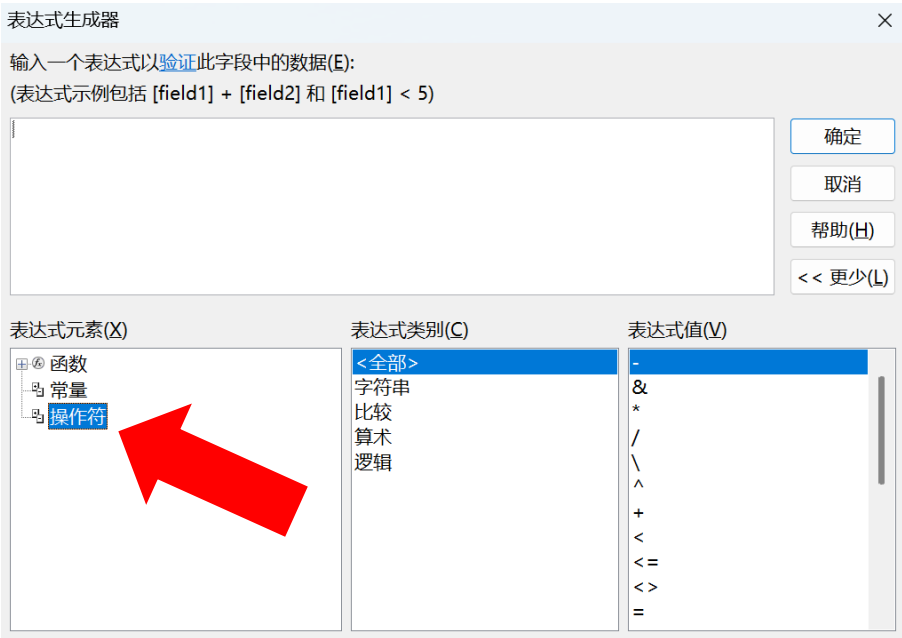
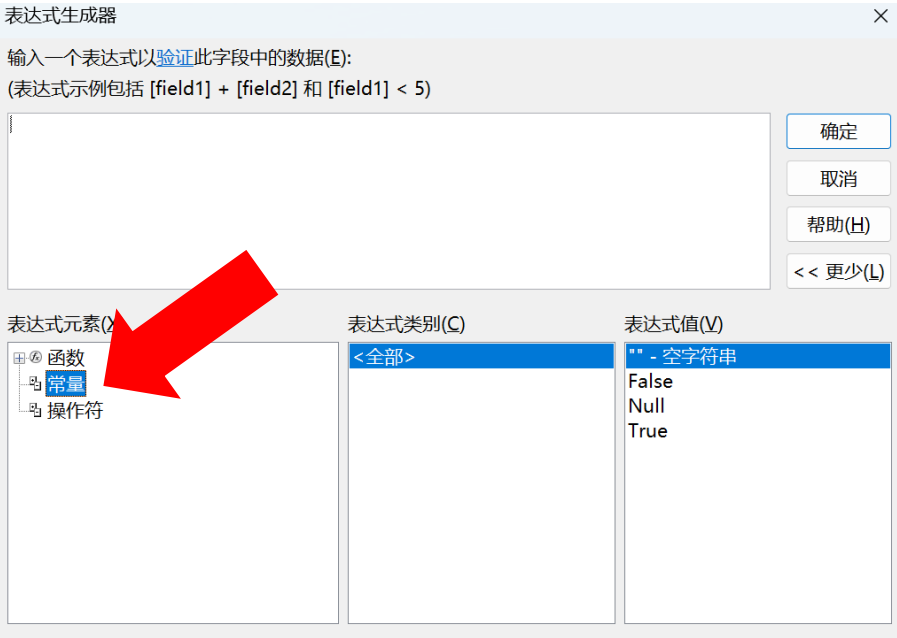
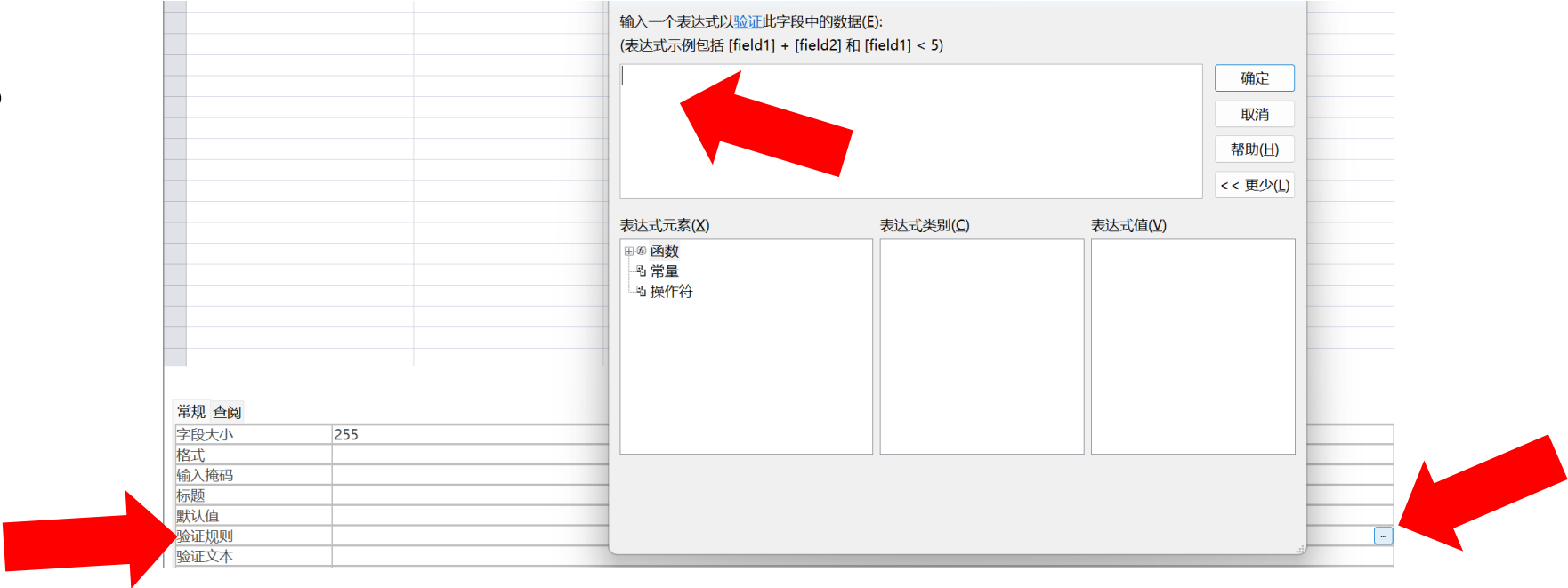
4) Add New Fields

5) Set Data Type

字段名称	数据类型
StudentID	短文本
Name	短文本
Age	短文本
Gender	短文本

字段名称	数据类型
StudentID	短文本
主键(K)	短文本
剪切(I)	数字
复制(C)	短文本
粘贴(P)	
插入行(I)	
删除行(D)	
属性(P)	

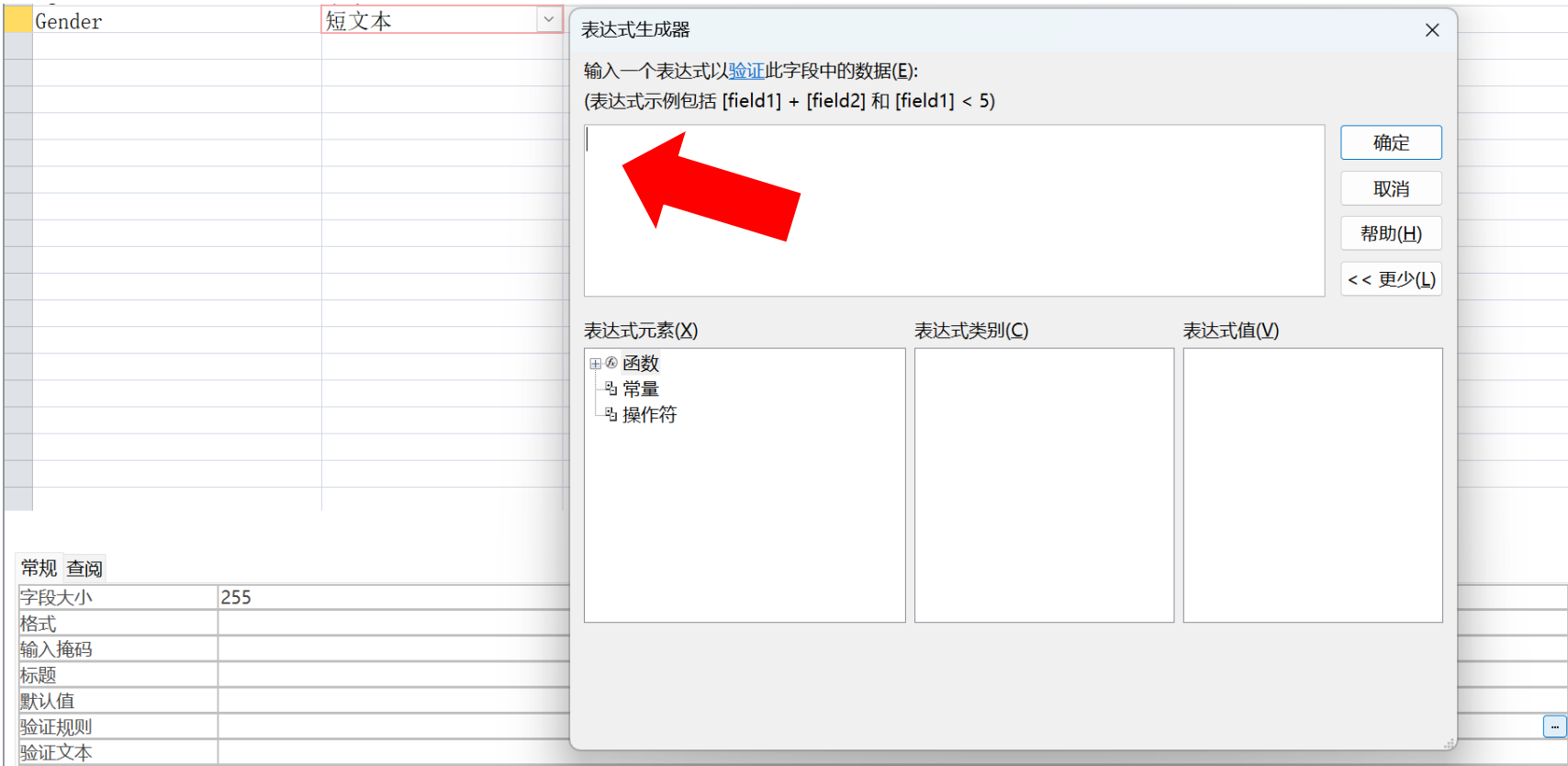
6) Set Constraints



BETWEEN 100 AND 1000
(与 ≥ 100 AND ≤ 1000 相同)

验证文本	年龄必须在15到30之间!
------	---------------

6) Set Constraints [Gender]



IN

测试值是否等于列表中的现有成员。比较值必须是括在圆括号中的逗号分隔列表。

IN ("东京","巴黎","莫斯科")

6) Set Constraints [Gender]

表达式生成器

×

输入一个表达式以验证此字段中的数据(E):
(表达式示例包括 [field1] + [field2] 和 [field1] < 5)

In("男","女")



确定

取消

帮助(H)

<< 更少(L)

验证规则	In ("男","女")
验证文本	请输入 '男' 或 '女'

7) Similar processes for “courses” Table

courses	
字段名称	数据类型
CourseID	短文本
CourseName	短文本
Credits	数字

- 课程表（courses）
 - 课程号（CourseID）
 - 课程名（CourseName）
 - 学分（Credits）

工具 | 显示/隐藏 | 字段、记录和表格事件

courses	
字段名称	数据类型
CourseID	短文本
CourseName	短文本
Credits	数字

表达式生成器

输入一个表达式以验证此字段中的数据(E):
(表达式示例包括 [field1] + [field2] 和 [field1] < 5)

>=1 And <=4

确定
取消
帮助(H)
<< 更少(L)

默认值	1
验证规则	>=1 And <=4
验证文本	学分通常为1~4!

8) Similar processes for “enrollments” Table

enrollments	
字段名称	数据类型
StudentID	短文本
CourseID	短文本
Grade	短文本

- 选课表（enrollments）
 - 学号（StudentID）
 - 课程号（CourseID）
 - 成绩（Grade）

主键

生成器

测试

验证规则

插入行

删除行

修改查阅

属性表

索引

创建数据宏

重命名/删除宏

显示/隐藏

字段、记录 and 表格事件

enrollments

字段名称	数据类型
StudentID	短文本
CourseID	短文本
Grade	短文本

主键

生成器

测试

验证规则

插入行

删除行

修改查阅

属性表

索引

创建数据宏

重命名/删除宏

显示/隐藏

字段、记录 and 表格事件

enrollments

字段名称	数据类型
StudentID	短文本
CourseID	短文本
Grade	短文本

8) Similar processes for “enrollments” Table

enrollments ×	
字段名称	数据类型
StudentID	短文本
CourseID	短文本
Grade	数字



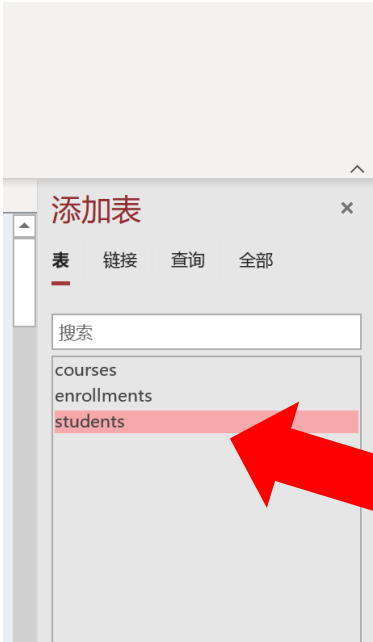
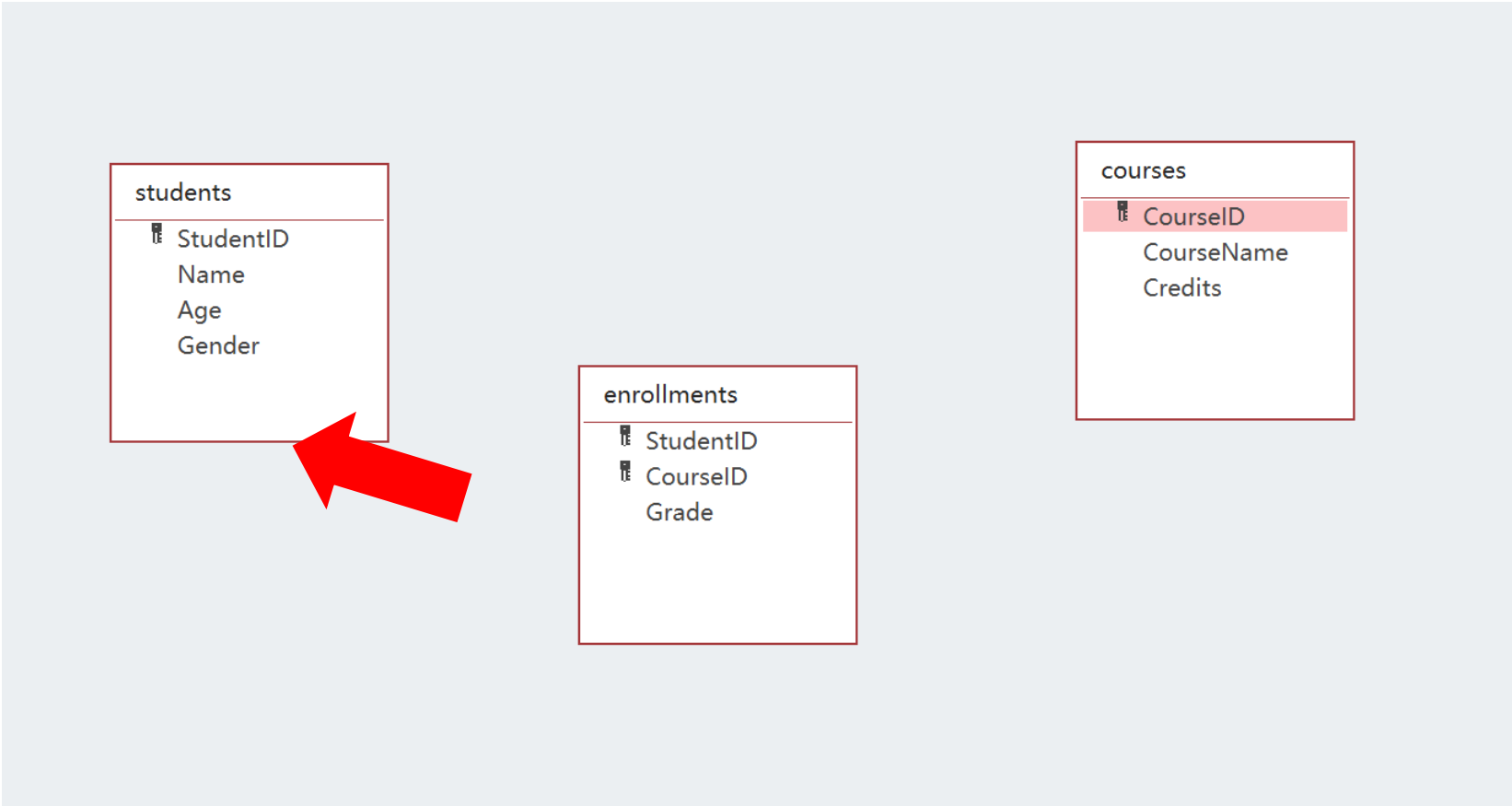
默认值	0
验证规则	>=0 And <=100
验证文本	分数为0~100!

9) Set Relationships Between Tables



9) Set Relationships Between Tables

Drag the tables to the operation area



9) Set Relationships Between Tables

Establish the relationship between the “students” table and the “enrollments” table

students	
	StudentID
	Name
	Age
	Gender

enrollments	
	StudentID
	CourseID
	Grade

编辑关系

表/查询(I):

students

相关表/查询(R):

enrollments

创建(C)

取消

联接类型(J)..

新建(N)..

StudentID	StudentID

☒ 实施参照完整性(E)

☐ 级联更新相关字段(U)

☐ 级联删除相关记录(D)

关系类型:

一对多

9) Set Relationships Between Tables

Establish the relationship between the “courses” table and the “enrollments” table

具

天系

关系

编辑关系

表/查询(I):
courses
CourseID
实施参照完整性(E)
级联更新相关字段(U)
级联删除相关记录(D)
关系类型:
一对多

相关表/查询(R):
enrollments
CourseID
新建(N)..

创建(C)
取消
联接类型(J)..

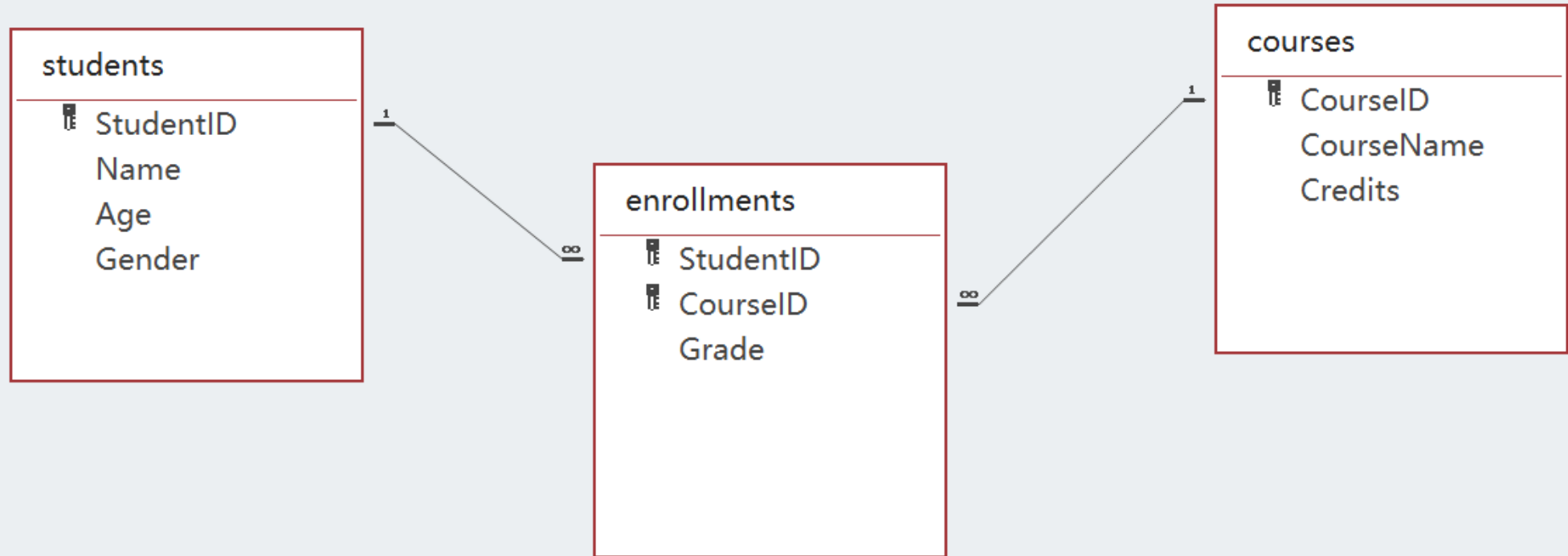
courses

CourseID
CourseName
Credits

enrollments

StudentID
CourseID
Grade

9) Set Relationships Between Tables



Integrity Constraint

2. Experiments

Correctly insert new student

INSERT INTO Students (StudentID, Name, Age, Gender) VALUES ('S004', '孙八', 20, '女');

	StudentID	Name	Age	Gender	单击以添加
+	S004	孙八	20	女	
*			0		

Correctly insert new course

INSERT INTO Courses (CourseID, CourseName, Credits) VALUES ('C002', '数据库原理', 3);

	CourseID	CourseName	Credits	单击以添加
+	C002	数据库原理	3	
*			1	

Correct enrollment (premise: both S004 and C002 exist)

INSERT INTO Enrollments (StudentID, CourseID, Grade) VALUES ('S004', 'C002', 88);

	StudentID	CourseID	Grade	单击以添加
+	S004	C002	88	
*			0	

1) Verify Entity Integrity (unique and non empty primary key)

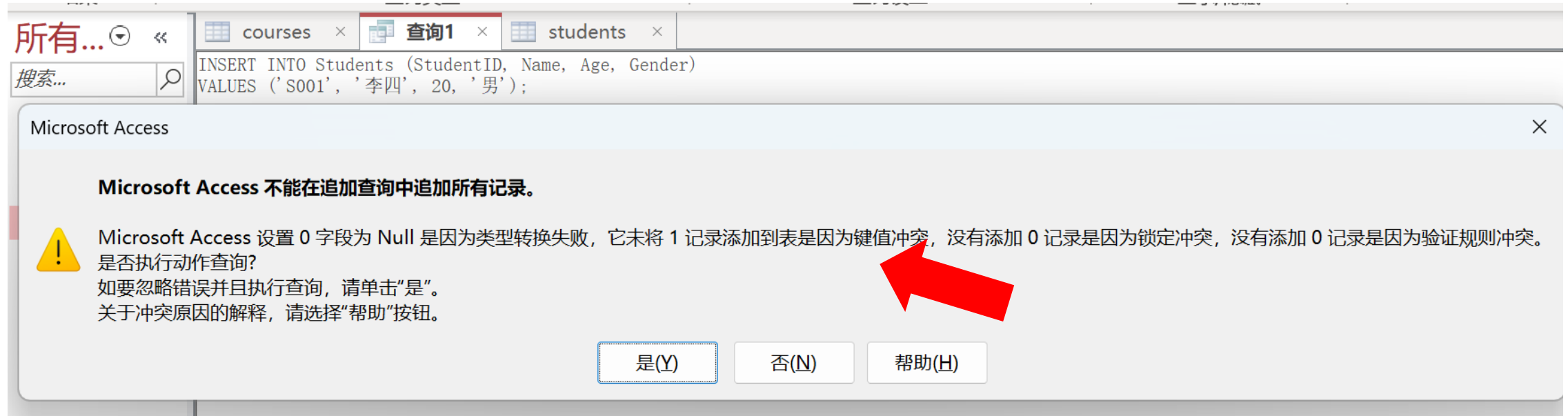
Add new data to “sutdents”

```
INSERT INTO Students (StudentID, Name, Age, Gender)  
VALUES ('S001', '李四', 20, '男');
```

students					
	StudentID	Name	Age	Gender	单击以添加
+	S001	李四	20	男	
+	S004	孙八	20	女	
米			0		

Insert duplicate data again (i.e. run the SQL statement again)

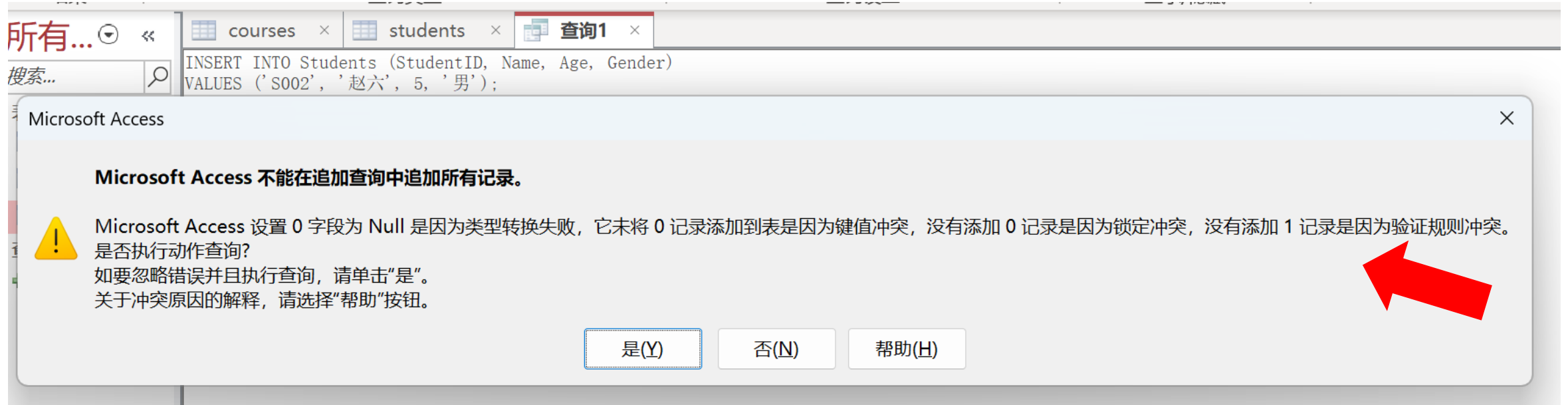
```
INSERT INTO Students (StudentID, Name, Age, Gender)  
VALUES ('S001', '李四', 20, '男');
```



2) Verify Domain Integrity

Inserting data beyond the age range

```
INSERT INTO Students (StudentID, Name, Age, Gender)
VALUES ('S002', '赵六', 5, '男');
```



2) Verify Domain Integrity

Inserting data of invalid gender

```
INSERT INTO Students (StudentID, Name, Age, Gender)
VALUES ('S003', '钱七', 22, '未知');
```



3) Verify Reference Integrity

Insert a non-existent student course selection record

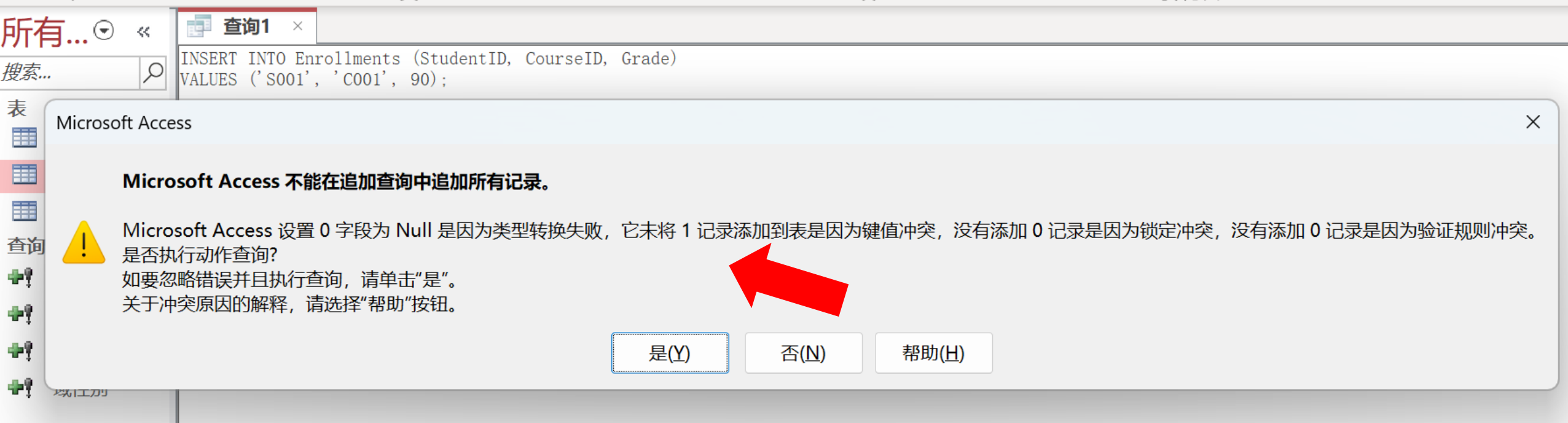
```
INSERT INTO Enrollments (StudentID, CourseID, Grade)  
VALUES ('S999', 'C004', 85);
```



3) Verify Reference Integrity

Insert a non-existent course

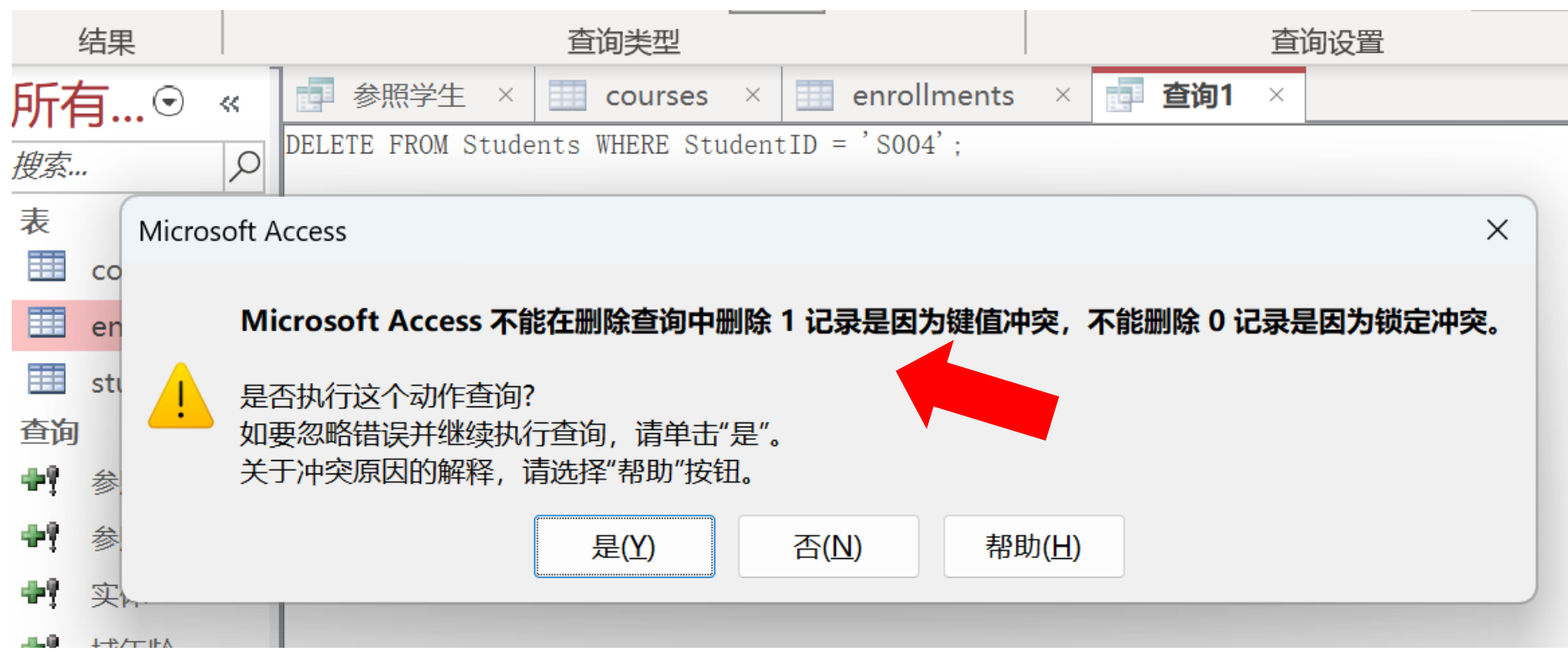
```
INSERT INTO Enrollments (StudentID, CourseID, Grade)
VALUES ('S001', 'C001', 90);
```



3) Verify Reference Integrity

Delete referenced students (recorded in Enrollments)

```
DELETE * FROM Students  
WHERE StudentID = 'S004';
```

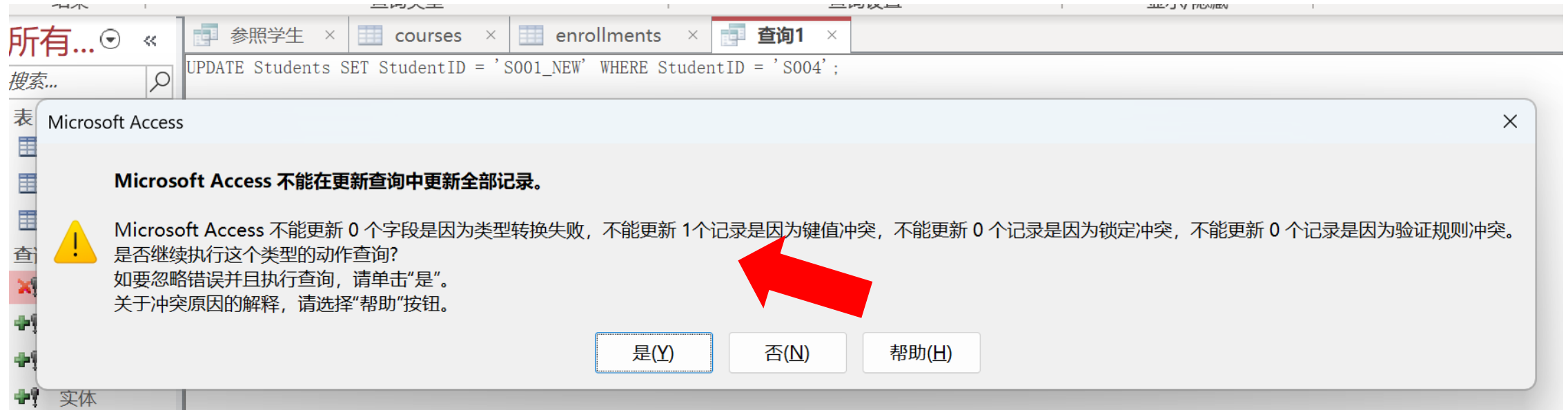


未启用“级联删除”, Access 会阻止删除, 因为相关记录存在于另一个表中。

3) Verify Reference Integrity

Update primary key of main table (referenced by foreign key)

```
UPDATE Students SET StudentID = 'S001_NEW'  
WHERE StudentID = 'S004';
```



未启用“级联更新”，会报错，因为存在相关记录。

Project 5

I. Tutorial on Integrity Constraint

II. Assignment

Assignment after Class

1. Content

- ✓ For Domain integrity constraint, Entity integrity constraint and Referential integrity constraint, you should give each integrity constraint “INSERT”, “DELETE” and “UPDATE” SQL statements to prove it.
- ✓ (At least 9 statements totally)

Assignment after Class

2. Data Schema

- The schema of the database is provided below (keys are red, field types are omitted):
 - student(**sid**, sname, sex, age, year, gpa)
 - dept(**dname**, numphds)
 - prof(**pname**, dname)
 - course(**cno**, cname, **dname**)
 - major(**dname**, **sid**)
 - section(**dname**, **cno**, **sectno**, pname)
 - enroll(**sid**, grade, **dname**, **cno**, **sectno**)

Assignment after Class

3. Data Files

- Download the following data files from WeChat Group
 - course.txt
 - dept.txt
 - enroll.txt
 - major.txt
 - prof.txt
 - section.txt
 - student.txt

Assignment after Class

4. Demand

- Hand in an ACCESS database that contains at least nine statements. The database should contain nine queries, named as follows:

Insert1

Insert2

...

Delete1

...

Update3

- Hand in a report which indicates your answers.(Figure is important. You should give your analysis about your sql statements.)